

MLIR NPU Compiler - Lecture 04 Worksheet

Topic: mlir-opt, mlir-translate, FileCheck - pass pipeline execution and regression testing

Exercise A - Command Anatomy

아래 command 의 각 부분을 설명하세요.

```
mlir-opt input.mlir --pass-pipeline='builtin.module(func.func(cse,canonicalize))' --mlir-print-ir-after-change
```

Part	Meaning
mlir-opt	
input.mlir	
builtin.module	
func.func	
cse,canonicalize	
--mlir-print-ir-after-change	

Exercise B - FileCheck Variables

아래 test 가 brittle 하지 않도록 CHECK line 을 보완하세요.

```
// RUN: mlir-opt %s --pass-pipeline='builtin.module(func.func(cse))' | FileCheck %s

func.func @simple_constant() -> (i32, i32) {
  %0 = arith.constant 1 : i32
  %1 = arith.constant 1 : i32
  return %0, %1 : i32, i32
}

// Write your CHECK-LABEL / CHECK-NEXT lines here:
// CHECK-LABEL:
// CHECK-NEXT:
// CHECK-NEXT:
```

Exercise C - NPU Command Ordering Test

DMA load -> barrier/await -> matmul -> DMA store 순서가 깨지지 않았는지 확인하는 FileCheck pattern 을 작성하세요.

```
// Expected output shape after npu-lower-to-command-buffer:
//   npu.dma.load ...
//   npu.barrier.await ...
//   npu.matmul ...
//   npu.dma.store ...

// CHECK-LABEL: func.func @command_order
// CHECK:
// CHECK:
// CHECK:
// CHECK:
```

Exercise D - Negative Diagnostic Test

tile_k 가 NPU systolic K granularity 의 배수가 아닌 경우 diagnostic 을 내는 테스트를 설계하세요.

```
// RUN: npu-opt %s --pass-pipeline='builtin.module(func.func(npu-tile{tile-k=7}))' -verify-diagnostics

func.func @negative_bad_tile_k(%a: tensor<128x256xf32>, %b: tensor<256x64xf32>) -> tensor<128x64xf32> {
  // expected-error@+1 {{tile-k must be a multiple of 16}}
}
```

```

%0 = "stablehlo.dot_general"(%a, %b) : (tensor<128x256xf32>, tensor<256x64xf32>) -> tensor<128x64xf32>
return %0 : tensor<128x64xf32>
}

```

Exercise E - Review Checklist

Question	Yes/No	Comment
CHECK line 이 pass 의 핵심 invariant 만 검증하는가?		
SSA 이름을 직접 고정하지 않았는가?		
순서가 correctness 인 command 에 CHECK-DAG 를 쓰지 않았는가?		
negative diagnostic case 가 있는가?		
pass pipeline anchor 가 의도한 operation level 과 맞는가?		